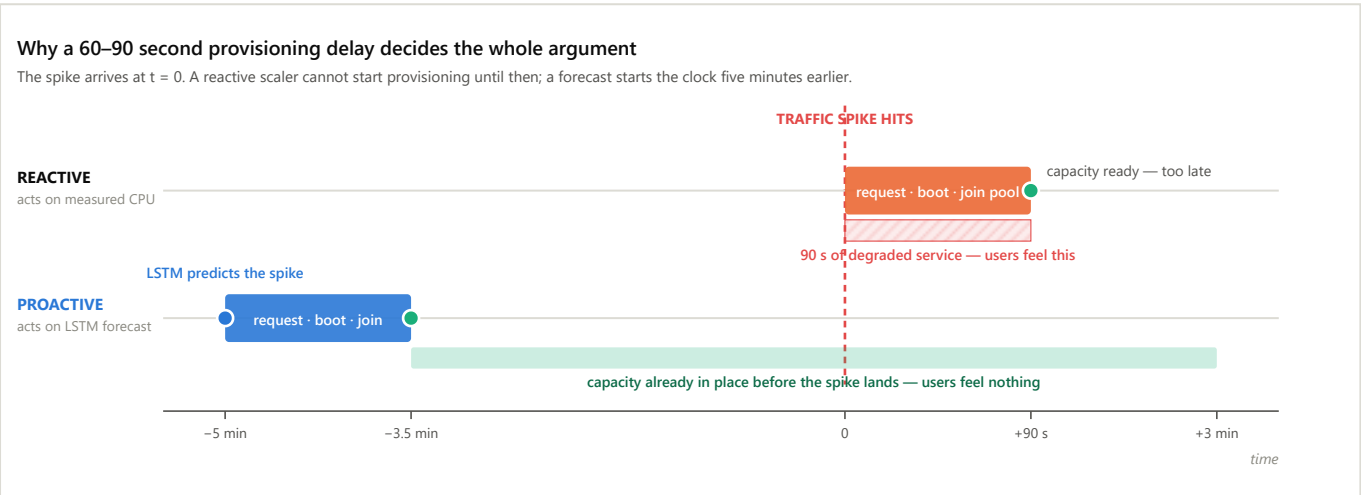


1 The idea, in one page

The project: predict a server's load 5 minutes ahead with an LSTM, so extra servers start **before** a spike instead of after it.

Why it is needed

- Cloud servers get sudden rushes of users. Normal auto-scaling **reacts**: it waits until CPU crosses a limit.
- A new server needs **60–90 seconds** to start. So help always arrives after users feel the slowdown.
- If we can **predict** the rush 5 minutes early, the new server is ready before it lands.



What must be proved at the end (Step 1.6)

One comparison: **proactive** scaling (uses the prediction) vs **reactive** scaling (uses the current reading), measured on **spike response time**, **wasted capacity** and **false alarms**.

Two phases

- **Phase 1** (now): full classical system — data → LSTM → scaling → dashboard.
- **Phase 2** (next semester): a 4-qubit quantum circuit is inserted between the LSTM and the final output layer, and compared against Phase 1. So Phase 1 must stay clean.

Where we are

STEP	WHAT	STATUS
1.1	Environment and folders	MOSTLY DONE README + git missing
1.2	Data: choose a VM, clean it	MOSTLY DONE synthetic generator missing
1.3	Prepare data for the model	DONE all checks pass
1.4	Build and train the LSTM	NEXT

2 The data, and choosing one VM

The dataset

- **GWA-T-12 Bitbrains**, fastStorage trace, published by TU Delft. Real servers of banks and insurance companies.
- **1,250 VMs**, one CSV each, semicolon-separated, one reading every **5 minutes**, covering one month (12 Aug – 11 Sep 2013).

From 11 raw columns to 4

OUR COLUMN	MADE FROM	WHY
datetime	Timestamp	Header says milliseconds but the values are seconds ; the code detects this.
cpu	CPU usage [%]	Already a percentage. Clipped to 0–100.
mem	$\text{used} \div \text{given} \times 100$	Raw memory is millions of KB; without converting, the model would treat memory as more important just because the numbers are bigger.
net	received + transmitted	Total activity is the signal; direction does not matter here.

Disk columns and the fixed allocation figures (core count, MHz capacity) are dropped — this project forecasts and scales only CPU, memory and network.

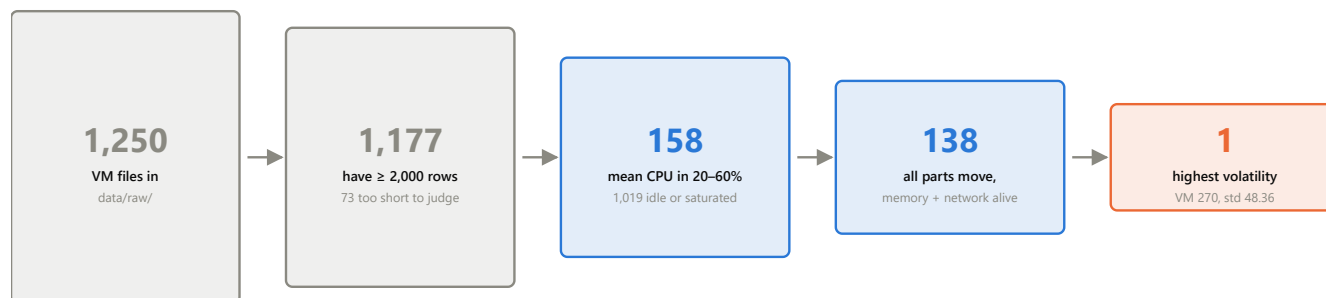
Why all three together: when a job starts, CPU, memory and network rise together. A model that sees all three has more clues than one that sees CPU alone.

Choosing a VM: five rules

- 1 **At least 2,000 readings** (~7 days) → 1,250 files become **1,177**.
- 2 **Mean CPU 20–60%**. Below that the VM is idle, above it is always full; both teach the model one constant answer. → **158** pass. (889 VMs, 76%, are below 5%.)
- 3 **Highest standard deviation first** — std measures how much the CPU actually moves, which is what there is to learn.
- 4 **All three parts must move** (train / validation / test).
- 5 **Memory and network must not be dead**. The model predicts all three metrics, so a metric stuck on one value would be a column with nothing to learn. A metric is rejected if it never changes, or if one value covers more than 90% of its readings. → caught **12** VMs with dead memory and **225** with dead network across the dataset.

→ **138** real candidates remain, and **VM 270** is top of that list.

VM selection funnel — 1,250 candidates down to one training workload



The 20–60% band is not arbitrary: below it the model learns to output a constant near zero, above it a constant near 100. Ranking by standard deviation asks “does its load actually move?”, and the last check makes sure it keeps moving in the test period too.

TWO FAULTS FOUND AND FIXED IN THE SURVEY

- **Switched-off days were counted.** An off VM still writes a row with every number at 0. The first pick, VM 214, looked like 44.21% mean only because of that; while running it was **68.43%** — outside the band. Fix: skip rows where the memory given to the VM is 0.
- **The test period was never checked.** The next pick, VM 200, had the highest std of all but its last 4.5 days are flat zero — exactly the slice kept for testing. Fix: rule 4 above.

The chosen VM: 270

PROPERTY	VALUE	MEANING
Mean CPU / std	46.81% / 48.36	Inside the band, top of the ranking
Switched-off rows	0	Ran the whole month
Daily rhythm	29% → 62%	Quiet at 11:00, busiest at 20:00
Shape of readings	47% / 36%	At 0–2% / above 98% — a batch machine, mostly on or off

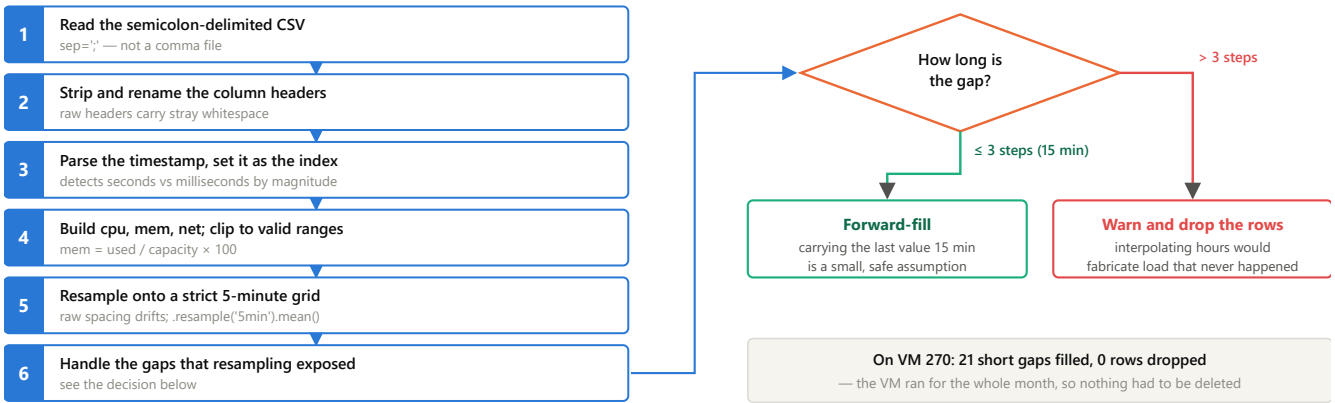
3 Cleaning the data

File: `utils/load_data.py` — one raw file in, a clean 4-column table out.

The six steps

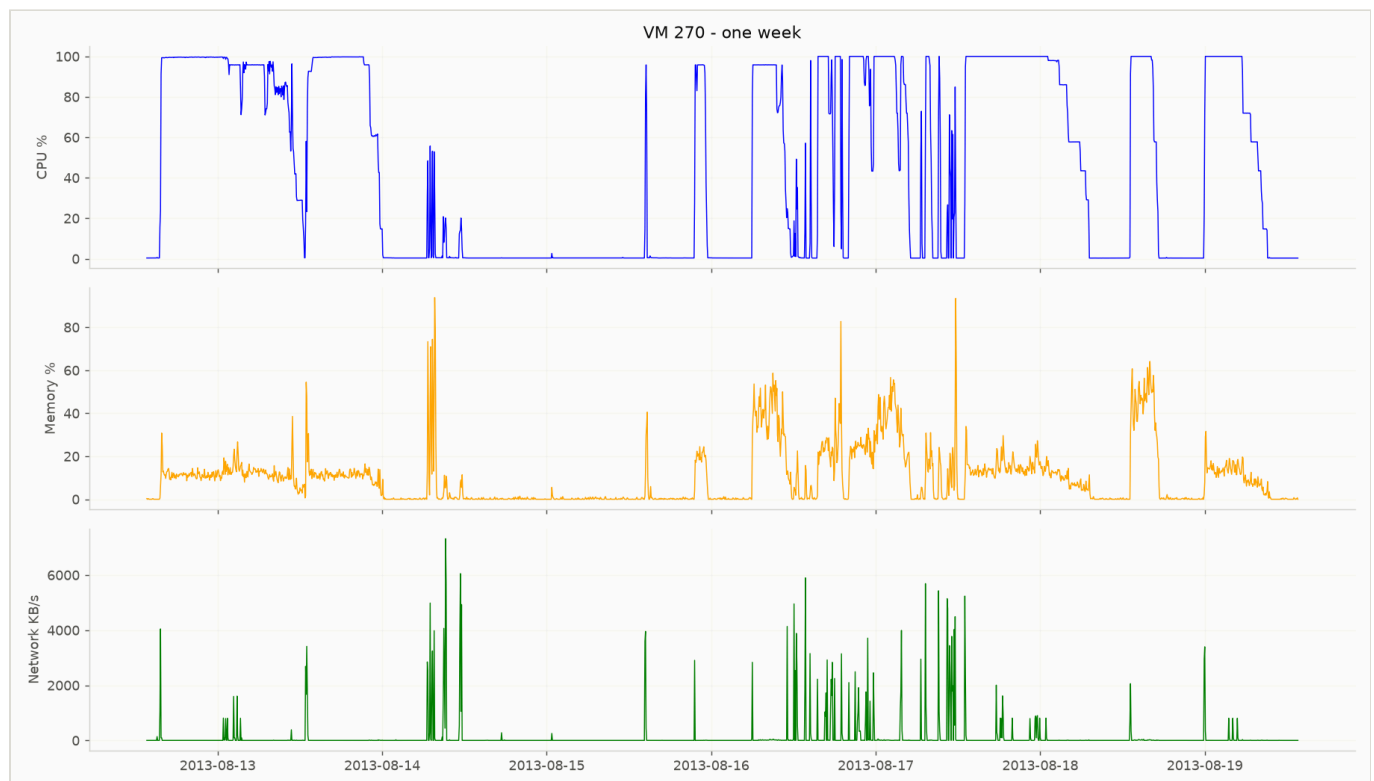
- 1 **Read with ; as separator** — these files are not comma-separated.
- 2 **Clean the column names** — the raw headers carry stray spaces.
- 3 **Parse the timestamp** and make it the index — detects seconds vs milliseconds by size.
- 4 **Build cpu, mem, net** and clip the percentages to 0–100.
- 5 **Resample onto an exact 5-minute grid** — raw spacing drifts, and the model needs every step to mean the same amount of time.
- 6 **Handle gaps:** fill up to 3 missing steps (15 min) with the last value; anything longer is **deleted, never guessed**, because inventing hours of load would teach the model something false.

load_vm() — six operations, and why the order matters



Result on VM 270

RESULT	VALUE	MEANING
Clean readings	8,640	Exactly 30.0 days
Short gaps filled	21	Holes of 15 minutes or less
Rows deleted	0	Nothing had to be thrown away
Time jumps	0	Every row exactly 5 minutes after the one before



One week of the clean data: CPU (blue), memory (orange), network (green).

4 Do the three metrics really move together?

This is the assumption behind feeding CPU, memory and network to one model, so it is worth proving rather than stating.

What "correlation" means

- A single number between **-1 and +1** that says how closely two things move together.
- **+1** = they always rise and fall together. **0** = no relationship. **-1** = one rises when the other falls.
- Rough reading: below 0.3 weak, 0.3–0.7 moderate, above 0.7 strong.

How the number is calculated

For every reading, check how far CPU is from its own average, and how far memory is from its own average. Multiply the two. If both are above average, or both below, the answer is **positive**. If one is above while the other is below, it is **negative**. Average all those products, then divide by the two standard deviations so the result always lands between -1 and +1.

VM 270's averages are **CPU 45.64%** and **memory 11.76%**. Taking one reading every 5 days, with no picking and choosing:

DATE	CPU	MEMORY	CPU - 45.64	MEM - 11.76	PRODUCT	SAME SIDE?
12 Aug	0.4	0.3	-45.3	-11.5	+521	yes — both low
17 Aug	100.0	14.4	+54.4	+2.6	+141	yes — both high
22 Aug	100.0	36.3	+54.4	+24.6	+1338	yes — both high
27 Aug	0.3	0.8	-45.3	-11.0	+498	yes — both low
1 Sep	0.3	0.0	-45.3	-11.8	+535	yes — both low
6 Sep	9.0	22.2	-36.6	+10.4	-381	no — opposite

Mostly positive products → a positive correlation. Doing this for all 8,640 readings gives **r = 0.7082**, which rounds to **0.71**. Computed two ways — by the formula above and by pandas `.corr()` — both give 0.7082.

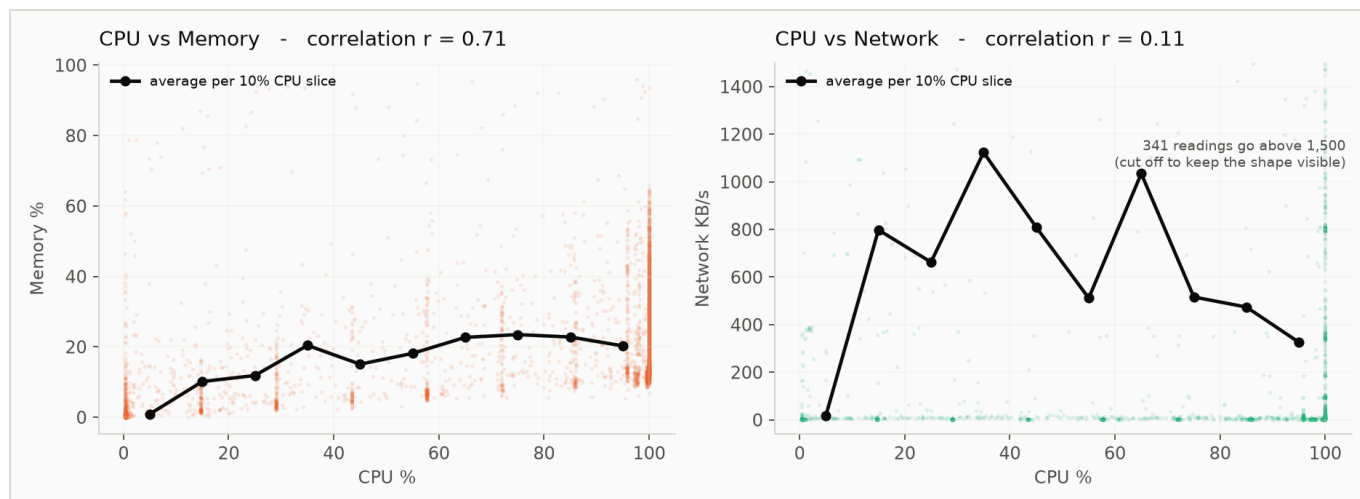
The three results

PAIR	R	READINGS ON THE SAME SIDE OF THEIR AVERAGE	MEANING
CPU & memory	0.71	90.2%	Strong — they really do move together
Memory & network	0.21	—	Weak
CPU & network	0.11	58.3%	Almost none — 50% would be pure chance

A second, simpler check — split every reading by how busy the CPU was:

VM 270 STATE	READINGS	AVERAGE MEMORY	AVERAGE NETWORK
CPU idle (below 2%)	4,087	0.7%	5.8 KB/s
CPU busy (above 98%)	3,065	23.3%	197.7 KB/s

Memory is **33×** higher when the CPU is busy. Network is **34×** higher on average too — but only in bursts, which is why its correlation stays low.



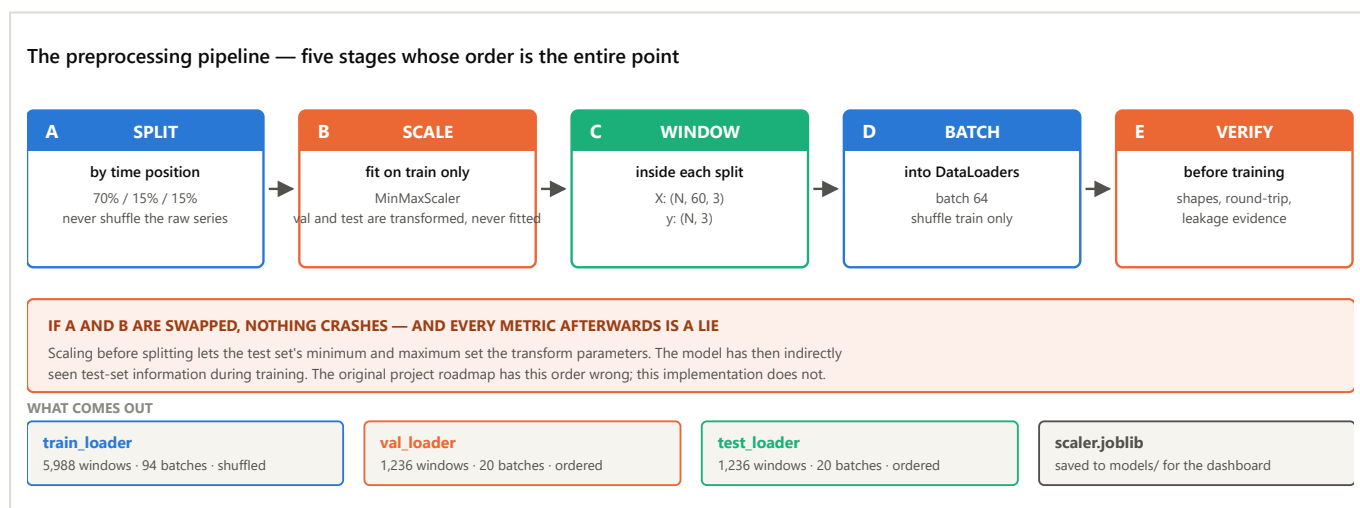
The same thing as a picture. Each dot is one 5-minute reading; the black line is the average of each 10%-wide CPU slice. **Left:** memory climbs steadily as CPU climbs — a real relationship. **Right:** the network line jumps up and down with no direction, which is what $r = 0.11$ looks like.

THE CONCLUSION TO STATE OUT LOUD

Memory genuinely follows CPU ($r = 0.71$), so feeding both to one model is justified. Network does not follow CPU on this VM ($r = 0.11$). It still belongs in the model, because the scaling engine must predict network load too, but we should expect it to be the hardest of the three to forecast — and we should say so before Step 1.5 rather than be surprised by it.

5 Preparing the data for the model

File: `utils/preprocessing.py` — **the order of these steps is the whole point.**



A — Split by time first (70 / 15 / 15)

- First 70% of the days trains, next 15% validates, last 15% tests. **Never random.**
- In real life we only know the past. Random splitting would let the model "predict" Monday after seeing Tuesday — the score would look great and mean nothing.

B — Scale to 0–1, fitted on training data only

- MinMaxScaler learns the min and max from **training** only, then transforms all three parts.
- If it also saw the test part, test information would leak into training and results would look better than they are.

- **Proof it worked:** training scales to exactly 0.00–1.00 but test reaches **1.91**. Going outside 0–1 is only possible if the scaler never saw the test data.
- Saved to `models/scaler.joblib`, so the dashboard can turn predictions back into real CPU %, memory % and KB/s.

C — Sliding windows, built inside each part

- One example = **60 readings (5 hours)** of all three metrics; the answer = the **next reading (5 minutes later)**. Then the window slides one step.
- 60 steps is long enough to contain a whole job starting and finishing, short enough to train quickly.
- 5 minutes ahead comfortably covers the 60–90 second startup delay.
- Windows are made separately per part, so no example mixes training and test data.

D — Batches of 64

PART	ROWS	WINDOWS	BATCHES	SHUFFLED?
Training (70%)	6,048	5,988	94	Yes — window order only
Validation (15%)	1,296	1,236	20	No
Test (15%)	1,296	1,236	20	No

Shuffling reorders whole 5-hour windows, never the readings inside them. Each part loses 60 windows because the first 60 rows can only be history.

E — Checks before any training

Windowing mistakes do not crash anything — they quietly produce data the model cannot learn from. So the pipeline checks itself:

CHECK	EXPECTED	RESULT
Input batch shape	(64, 60, 3)	PASS
Answer batch shape	(64, 3)	PASS
Un-scale window 0, compare with the CSV	difference $\approx$ 0	PASS 0.00024 (rounding only)
Test data allowed outside 0–1	yes	PASS up to 1.91

STILL TO DO BEFORE STEP 1.4, AND THE LIMITS TO ADMIT

To do: `README.md` · `git init` and a first commit · `utils/generate_data.py` (synthetic data with spikes at known times, needed for the Step 1.6 comparison).

Limits: VM 270 is a batch machine (mostly 0% or 100%), which is harder to predict than a smooth curve. The top candidates are near-copies of each other. Only one VM and one month of data are used.